ALGORITHM

Spring 2026, Midterm Exam, April 15

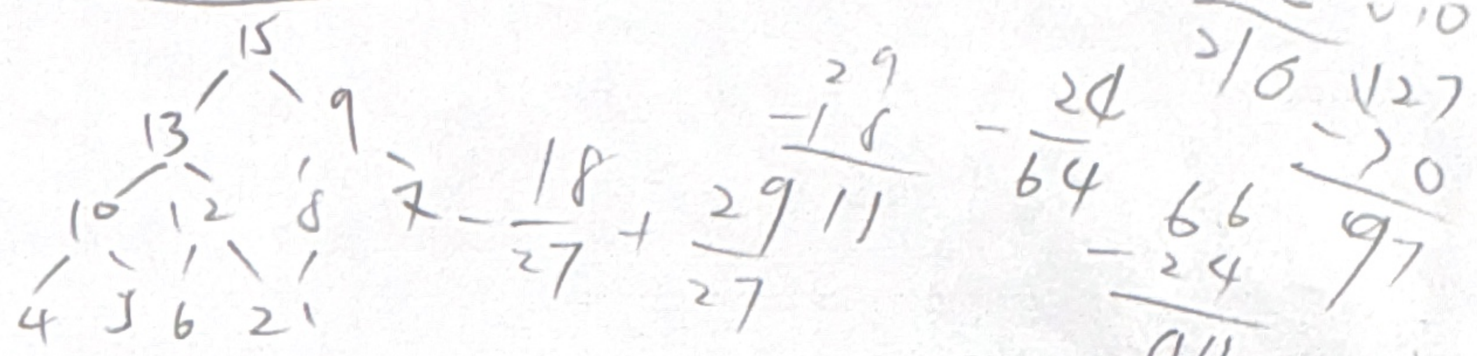
5:00 – 8:00 pm

1. (15%) Illustrate the operation of the following sorting algorithms respectively on the array $A = \langle 3, 7, 1, 4, 0, 6, 4, 2, 1, 3, 4 \rangle$, where $A[j] \in \{0, 1, \ldots, 7\}$ for $1 \le j \le 11$. Which of them are in-place sorting algorithms?

   (a) MERGE SORT

   (b) INSERTION SORT

   (c) QUICKSORT



2. (30%) Please answer the following questions.

   (a) Show that any binary tree of height $h$ has at most $2^h$ leaves.

   (b) Use the master method to give a tight asymptotic bound on the running time of the recurrence $T(n) = 8T(n/2) + n^3$.

   (c) Illustrate the operation of MAX-HEAP-INSERT$(A, 3)$ on the heap $A = \langle 15, 13, 9, 10, 12, 8, 7, 4, 5, 6, 2, 1 \rangle$.

   (d) Finding tight bound for the function $f(n) = n^3 - 6n + 2$ by using the $\Theta$ notation. Justify your answer by demonstrating the constants.

   (e) Show that $n^3 - 6n + 2 \neq \Omega(n^4)$.

   (f) The operation HEAP-DELETE$(A, i)$ deletes the item in node $i$ from heap $A$. Give an implementation of HEAP-DELETE that runs in $O(\lg n)$ time for an $n$-element max-heap. You may utilize any standard heap operations discussed in class, such as MAX-HEAPIFY or HEAP-INCREASE-KEY.

3. (20%) Use a recursion tree to determine a good asymptotic upper bound on the recurrence $T(n) = 4T(\lfloor n/4 \rfloor) + 2n$, where $T(1) = 2$, $T(2) = 5$ and $T(3) = 8$. Use the substitution method to verify your answer.

4. (20%) Given an array $A = \langle x_1, x_2, \ldots, x_n \rangle$ of numbers, the minimum-subarray problem is to find two indices $i$ and $j$ with $1 \le i \le j \le n$ such that the contiguous subarray $\langle x_i, x_{i+1}, \ldots, x_j \rangle$ has the smallest possible sum among all contiguous subarrays of $A$.

   (a) Illustrate the operation of FIND-MINIMUM-SUBARRAY on the array $A = \langle -3, 5, -3, -4, -2, 2, 4, -3 \rangle$.

   (b) Describe an algorithm that solves the minimum-subarray problem in $O(n \log n)$ time.

5. (15%) You are a fisherman trying to catch fish using a net of width $d$ meters. Using advanced technology, you know that the positions of all $n$ fish in the sea are integers on a number line. Multiple fish may occupy the same position. To catch fish, you cast your net at position $x$, which captures all fish with positions in the interval $[x, x+d]$ (**inclusive**). Given $n$, $d$, and an array $A[1, \ldots, n]$ representing the positions of the fish, design an $O(n \log n)$ algorithm to compute the maximum number of fish that can be caught with a single cast.

For example, if $n = 7$, $d = 3$, and $A = [1, 11, 4, 10, 6, 7, 7]$, then the maximum number of fish that can be caught is 4. This is achieved by casting the net at $x = 4$, which captures fish at positions 4, 6, and two fish at position 7.

6. (15%) Suppose that $A$ is an array of $n$ elements. Each element of $A$ has some flavor; you cannot tell the flavors apart, but you know that one flavor is a **strict majority**. That is, there are strictly more than $n/2$ elements that have this one flavor. You have access to a genie called IsMajority so that IsMajority$(A, x)$ returns **True** if $x$ is an element of $A$ with the majority flavor, and **False** otherwise.

You develop the following algorithm to return an element of the majority flavor, when $n$ is a power of 2:

```
Find-Majority(A)
1  n = length[A]
2  if n == 1
3        return A[1]
4  a = Find-Majority(A[1..n/2])
5  b = Find-Majority(A[n/2+1..n])
6  if Is-Majority(A, a)
7        return a
8  else return b
```

(a) Use induction to prove that Find-Majority correctly returns an element of the majority flavor. *Hint: Consider how the majority flavor in the global array relates to the majority flavors in the left and right subarrays.*

(b) Suppose Is-Majority takes $O(1)$ time. Write a recurrence relation for the running time $T(n)$ of Find-Majority and solve it to determine its asymptotic complexity using the Master Theorem or a recursion tree.